

错题本

WA / RE / TLE 排查速查手册

打印用 • 比赛翻阅 • 提交前过一遍

2026 年 7 月 23 日

目录

第一章 板子	5
第二章 提交前速查	6
第三章 通用 WA 错误	7
3.1 溢出	7
3.2 多测	7
3.3 取模	7
3.4 边界	7
3.5 输出格式	8
3.6 杂项	8
3.7 15 分钟还没找到 WA	8
第四章 错题复盘与最小反例	9
4.1 一次 WA 的完整复盘流程	9
4.2 多测与 <code>void solve()</code> : 最容易被忽略的真实错误	9
4.2.1 中途 <code>return</code> 会把下一组输入读乱	10
4.2.2 全局容器和节点池必须按题目边界清理	10
4.2.3 多测回归样例的固定格式	11
4.3 边界样例设计 SOP	11
4.3.1 先列“变化维度”，再写数据	11
4.3.2 每个大型板子至少四组核心测试	12
4.4 最小反例库：按错误类型直接查	12
4.4.1 下标与半开区间	12
4.4.2 前缀和与差分	12

4.4.3	图论：不可达、重边、自环	13
4.4.4	最短路：旧堆项和不可达点	13
4.4.5	动态规划：全负、空转移和答案位置	13
4.4.6	线段树：整段标记后查询子区间	13
4.4.7	字符串：重叠和周期	14
4.4.8	计算几何：相切、共线和重复点	14
4.5	错误代码与修复代码对照	14
4.5.1	负数取模	14
4.5.2	区间最小值的错误懒标记	14
4.5.3	LCA 根节点初始化	15
4.6	可直接改名使用的对拍框架	15
4.7	错题卡：以后每道题只记录一页	16
4.8	提交前五分钟固定动作	16

第五章 通用 RE 错误 17

5.1	RE 排查流程	17
5.2	数组越界	17
5.3	栈溢出（递归过深）	18
5.4	除以零 / 取模零	18
5.5	非法内存访问	18
5.6	STL 误用	18
5.7	整数未定义行为	19
5.8	常见 RE 清单速查	19

第六章 图论与树 20

6.1	建图	20
6.2	特殊图	20
6.3	DFS / BFS	20
6.4	最短路	20
6.5	拓扑排序	21
6.6	LCA / 树形 DP	21
6.7	图论模型先判型	21

6.8	连通块、0-1 BFS 与最小生成树	21
6.8.1	连通块：外层循环不能漏	21
6.8.2	0-1 BFS：权值必须真的只有 0 和 1	22
6.8.3	Kruskal：最小生成树的三个返回值	23
6.8.4	强连通分量、割点和桥：最容易混淆的地方	24
6.9	图论回归样例：四组就能抓住大多数板子错误	24
第七章	动态规划	25
7.1	初始化	25
7.2	遍历顺序	25
7.3	转移遗漏	25
7.4	边界	25
7.5	先写状态三句话，再写循环	26
7.6	遍历顺序：是否会偷偷重复使用	26
7.7	不可达状态：不能把 INF 当成普通数字	27
7.8	滚动数组：清空的是“当前层”	27
7.9	答案位置、路径恢复与计数取模	27
7.10	DP 最小回归样例	28
第八章	数据结构	29
8.1	并查集	29
8.2	线段树	29
8.3	二分	29
8.4	双指针 / 滑窗	29
8.5	树状数组：下标、前缀和与差分	30
8.5.1	区间加、单点查：把差分藏进树状数组	30
8.6	单调栈与单调队列	30
8.6.1	下一个更大元素：栈内保存什么	30
8.6.2	滑动窗口最小值：队首才是答案	31
8.7	离散化与区间结构	31
8.8	数据结构回归样例	32

第九章 博弈论	33
9.1 必胜态与必败态	33
9.2 Bash 游戏：先找模周期	33
9.3 普通 Nim：异或和为零就是必败	34
9.4 Sprague-Grundy 函数与 mex	34
9.5 反常 Nim、阶梯 Nim 与常见变形	35
9.6 Minimax 极大极小搜索与 Alpha-Beta 剪枝	35
9.7 博弈题的现场检查顺序	36
9.8 博弈回归样例	36
第十章 数学与精度	38
10.1 取模（再强调）	38
10.2 组合计数	38
10.3 数论	38
10.4 浮点	38
第十一章 计算几何	39
11.1 精度	39
11.2 向量基础	39
11.3 线段与直线	39
11.4 凸包	40
11.5 多边形	40
11.6 圆	40
11.7 旋转卡壳	40
11.8 半平面交	40
11.9 常见坑总结	41
第十二章 读题错误	42
12.1 数值范围	42
12.2 核心对象	42
12.3 操作规则	42
12.4 坐标与格式	42

1 板子

比赛开局直接粘贴的模板。注意 `#define int long long` 之后 `main` 必须写 `signed main`。

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int inf = 2e18;
const int N = 1e6 + 10;

void solve() {
    int n;
    cin >> n;
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

2 提交前速查

大多数 WA 不是算法错了，是细节把正确思路拖下水。提交前花 30 秒过一遍这页。

1. 有没有 `1 << i` 没写成 `1LL << i`?
2. `inf + x` 会不会爆？转移前加 `if(dist[u] != inf)`。
3. 两个大数相乘有没有步步取模？
4. 多测的数组/图清空了吗？`solve()` 里有没有提前 `return` 导致输入没读完？
5. 减法取模补 `+ MOD` 了吗？除法用逆元了吗？
6. $n = 0, 1, 2$ 、全相同、全负数、无解、不连通测过了吗？
7. 输出大小写对吗？下标是 0-based 还是 1-based？行末空格？

3 通用 WA 错误

3.1 溢出

- $1 \ll i$: 常数 1 是 32 位, $i \geq 31$ 就炸。写 $1LL \ll i$ 。
- $\text{inf} + x$: $2 \times 10^{18} + x$ 爆 long long 变负数。松弛/转移前判不可达。
- 大数乘法: $10^{18} \times 10^{18}$ 直接爆。用 `__int128` 或步步取模。
- $\text{pow}(5,2)$ 可能返回 24.999..., 强转 int 变 24。整数幂用快速幂。

3.2 多测

- 串流: `solve()` 里提前 return, 这组数据没读完, 后面全乱。
- 残留: 上一组 $n = 10$, 这组 $n = 5$, 只清 1 ~ 5 留下 6 ~ 10 的脏数据。
- 重名: 全局 `int n;` 和局部 `int n;` 同时存在, DFS 读到的是全局的 0。

3.3 取模

- 减法: $(a - b \% \text{MOD} + \text{MOD}) \% \text{MOD}$, 防负数。
- 除法: 不能直接除, 要乘逆元 $a * \text{qpow}(b, \text{MOD}-2) \% \text{MOD}$ 。
- 连乘: $A * B \% \text{MOD} * C \% \text{MOD}$, 步步取模防中间爆。

3.4 边界

- $n = 0, 1, 2$ 特判。
- 全相同、全为 10^9 、全为负数。

- 图：不连通、链、菊花、自环、重边。
- 无解时输出什么？-1? 0? Impossible?

3.5 输出格式

- YES/Yes/yes 大小写。
- 下标 +1 还是不 +1。
- 行末空格、最后换行。
- 多解时要不要字典序最小。

3.6 杂项

- 贪心假了：无法证明就怀疑，考虑 DP / 二分答案。
- 比较器：sort 的 cmp 用了 $\leq$ 会破坏严格弱序，必须用 $<$ 。
- 未初始化：本地对但提交 WA，检查局部变量有没有赋初值。
- 哈希被卡：unordered_map 在 CF 上容易被 hack，加 custom hash 或换 map。

3.7 15 分钟还没找到 WA

1. 停止看代码。手造小样例（含负数、0、1、乱序）。
2. 纸上跑一遍算法。
3. 写暴力（ $O(n^3)$ 也行），写随机数据生成器，对拍。
4. 找到最小错误数据，定位第一处不一致。

4 错题复盘与最小反例

这一章不是再罗列“某某算法会错”，而是把一次错误真正变成下一次不会再犯的检查项。每道题至少留下四样东西：

1. 最小错误输入：尽量短，但必须仍然能复现错误；
2. 错误原因：指出哪一个不变量、边界或题意被破坏；
3. 修复后的关键代码：只保留真正改变结果的几行；
4. 回归测试：以后每次改模板都重新运行，防止旧错误回来。

4.1 一次 WA 的完整复盘流程

1. 先确认输入输出没有错：样例是否完整复制，是否漏读了 T 、多组边、空行或字符串；输出大小写、空格和换行是否符合题面。
2. 再缩小数据：删除无关点、边、操作，优先保留第一个产生错误的操作。一个能在 4 个数上复现的错误，不要拿 4 万个数解释。
3. 写出不变量：例如堆顶是当前最短距离、线段树节点的和对应整个区间、动态规划状态已经处理完前缀、回滚前后的历史栈长度相同。
4. 对照错误发生前后：在第一次不符合不变量的位置打印变量，而不是只打印最终答案。
5. 写进回归样例：把这个最小输入和正确输出放入模板旁边，下一次修改代码时必须重新跑。

不要一看到 WA 就重写整份代码。先找到“第一个错误状态”，否则新代码即使过了当前样例，也不知道究竟修复了什么。

4.2 多测与 `void solve()`：最容易被忽略的真实错误

4.2.1 中途 return 会把下一组输入读乱

下面的代码在无解时直接返回，当前测试剩余的边仍在输入流中，下一组的 n, m 就会读成上一组的边：

```
void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        if (u == v) return; // 错：剩余输入没有读完
    }
    cout << "YES\n";
}
```

如果输入是：

```
2
3 2
1 1
1 2
1 0
```

第一组提前返回后，第二组的 n, m 可能被读成 1,2，错误会看起来像“第二组算法随机坏了”。正确写法是读完当前组，再用标志记录非法状态：

```
void solve() {
    int n, m;
    cin >> n >> m;
    bool ok = true;
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        if (u == v) ok = false;
    }
    cout << (ok ? "YES" : "NO") << '\n';
}
```

4.2.2 全局容器和节点池必须按题目边界清理

多测最小测试：第一组规模大，第二组规模小。它可以专门暴露“只清理前 n 个位置”或“节点编号没有归零”：

```
2
5
```

```
1 2 3 4 5
1
9
```

如果第二组的答案仍然包含第一组的数据，优先检查：

- `vector` 是否调用了 `clear()` 或重新赋值；
- 图的边表、`head`、`edge_count` 是否一起清空；
- 字典树、主席树、回文自动机的 `tot` 是否恢复到根节点之后；
- 线段树的 `sum`、`lazy`、`tag` 是否全部重新 `build`；
- 时间戳数组是否使用了新的 `stamp`，而不是只清空一半下标。

4.2.3 多测回归样例的固定格式

以后给任何板子补样例时，优先使用下面的四组结构：

1. $n = 1$ 或空结构：检查根、叶子、空集合和初始化；
2. 退化结构：链、星、全相同、全负、无边或单条边；
3. 正常结构：至少有两条竞争路径、两种状态或两个不同区间；
4. 两组连续测试：第一组故意较大，第二组很小，检查状态污染。

4.3 边界样例设计 SOP

4.3.1 先列“变化维度”，再写数据

不要随机生成一堆看起来很大的数。先列出模板真正依赖的变化维度：

- 数组：长度 0, 1, 2、重复值、负数、最大值、单调和逆序；
- 区间：单点、整段、相邻、不相交、完全包含、端点重合；
- 图：无边、链、星、环、重边、自环、不连通、负边；
- 树：单点、链、星、平衡树、根是答案、根不在查询路径；
- 字符串：空串、单字符、全相同、周期串、重叠匹配、无匹配；
- 几何：重合、相切、两点交、平行、共线、退化多边形；
- 多测：大组后小组、空组后正常组、节点池复用。

4.3.2 每个大型板子至少四组核心测试

建议每个超过十行的模板都保留以下四组，而不是只保留随机样例：

类型	目标	典型输入
边界一	空结构或最小规模	$n = 0, 1$ 、单点、空边集
边界二	退化或等号分支	全相同、相切、重复边、 $x = \text{second_max}$
正常一	核心转移真的发生	两条竞争路径、重叠区间、至少一次旋转/合并
正常二	多阶段组合	先修改再查询、先连边再断边、多个版本交叉查询

如果板子存在随机化或浮点误差，再增加一组“同答案不同顺序”的测试：输入随机打乱后，答案允许顺序不同，但数值和不变量必须相同。

4.4 最小反例库：按错误类型直接查

4.4.1 下标与半开区间

所有只处理 $[l, r)$ 的代码都要测试 $l=0, r=1$ 、 $l=0, r=n$ 和相邻区间：

```

数组：      [10, 20, 30]
区间一：    [0, 1)  只包含 10
区间二：    [1, 3)  包含 20, 30
区间三：    [0, 3)  包含全部元素
区间四：    [1, 2)  只包含 20
  
```

如果区间三答案少一个元素，通常是把右端点当成闭区间；如果区间一访问到 $a[1]$ ，通常是把右端点错误地包含进来了。

4.4.2 前缀和与差分

对数组 $[1, 2, 3]$ 依次执行 $\text{add}(1, 1, 5)$ 、 $\text{add}(2, 3, -2)$ ，最终应为 $[6, 0, 1]$ 。这个样例同时检查：

- 单点更新是否正确；
- 负数更新是否正确；
- $r+1$ 是否写在正确位置；
- 查询 $[1, 3]$ 是否为 7，而不是只返回最后一次更新。

4.4.3 图论：不可达、重边、自环

```
4
1 0
2 1 1 2
2 2 1 2 1 2
3 3 1 1 1 2 2 3
```

四组分别测试单点、单边、重边和自环。不要默认题目没有自环；如果算法是桥、割点、强连通分量或拓扑排序，自环的处理完全不同：自环会让有向图出现环，但不会把无向单点自动变成桥。

4.4.4 最短路：旧堆项和不可达点

```
3 3
1 2 10
1 3 1
3 2 1
```

从 1 到 2 的答案是 2。Dijkstra 会先把 2 以 10 放进堆，再通过 3 改成 2；弹出旧的 (10,2) 时必须跳过。再增加一个没有任何入边的点，检查不可达点是否保持 INF ，而不是参与 $\text{INF}+w$ 的溢出运算。

4.4.5 动态规划：全负、空转移和答案位置

最大子段和的回归样例：

```
4
1
-5
3
1 -2 1
4
-3 -2 -5 -1
5
2 -1 2 -1 2
```

答案分别是 -5,1,-1,4。如果把 dp 初值写成 0，第一组就会错误；如果只在转移时更新答案而不检查最后一个位置，第四组也容易错。

4.4.6 线段树：整段标记后查询子区间

初始数组 [1,1,1,1]，先对 [1,4] 加 5，再查询 [2,3]，答案必须为 12；随后对 [2,2] 加 -3，再查整段，答案为 18。这个顺序专门验证 `push_down`：只查整段时懒标记可能一直藏着，只有访

问子区间才会暴露错误。

4.4.7 字符串：重叠和周期

文本 abababa、模式 aba 的匹配起点为 0,2,4，总数为 3；文本 aaaaaa、模式 aaa 的总数也为 3。匹配成功后不能把模式指针直接归零，必须回到前缀函数给出的最长可复用前缀。

4.4.8 计算几何：相切、共线和重复点

- 两个半径为 1 的圆，圆心距 2：一个交点；圆心距 1：两个交点；圆心距 3：无交点。
- 三点 (0,0),(1,0),(3,0)：外接圆应退化成最远两点的直径圆，不能除以零。
- 最近点对中加入两个相同点：答案必须为 0，不能用一个正的初始答案掩盖重复点。

4.5 错误代码与修复代码对照

4.5.1 负数取模

```
// 错误：C++ 的负数取模结果仍可能为负
long long add = (a - b) % MOD;

// 修复：先把减数归一化，再整体加模
long long add = (a - b % MOD + MOD) % MOD;
```

回归输入：a=3,b=5,MOD=7，答案应为 5，而不是 C++ 直接取模得到的 -2。

4.5.2 区间最小值的错误懒标记

```
// 错误：把 chmin 当成加法，直接给整个区间加 tag
tree[p].mx = min(tree[p].mx, x);
tree[p].sum += x * len; // 完全错误

// 修复思路：只有当前最大值会下降，
// 必须维护最大值、次大值、最大值出现次数和区间和。
if (second_mx[p] < x && x < mx[p]) {
    sum[p] -= (mx[p] - x) * cnt_mx[p];
    mx[p] = x;
}
```

回归数组 [1,3,3,7]：执行整段 chmin(5) 后和为 12，再执行 [2,3] chmin(2) 后该区间和为 4。第二次操作不能只看根节点最大值，必须按次大值条件决定是否继续递归。

4.5.3 LCA 根节点初始化

```
// 错误：根的父亲没有明确设为 0，倍增表含有旧数据
dfs(root, parent[root]);

// 修复：每组数据先固定根和所有根祖先
parent[root] = 0;
depth[root] = 0;
for (int j = 0; j < LOG; ++j) up[root][j] = 0;
dfs(root, 0);
```

回归树：单点、链 1-2-3-4、星 1-{2,3,4}，分别查询 $\text{lca}(1,1)$ 、 $\text{lca}(2,4)$ 、 $\text{lca}(3,4)$ 。根表没有清空时，单点组往往会错误继承上一组根的祖先。

4.6 可直接改名使用的对拍框架

下面的框架不替代暴力解，它只负责反复生成小数据、运行标准程序和暴力程序，并在第一次不一致时停下。把三个可执行文件名替换成自己的文件即可：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    for (int tc = 1; tc <= 100000; ++tc) {
        // generator.exe 负责把一组随机数据写入 input.txt
        system("generator.exe > input.txt");
        system("std.exe < input.txt > std.out");
        system("brute.exe < input.txt > brute.out");
        ifstream a("std.out"), b("brute.out");
        string x, y;
        bool same = true;
        while (a >> x || b >> y) {
            if (x != y) { same = false; break; }
            x.clear(); y.clear();
        }
        if (!same) {
            cerr << "Mismatch on test " << tc << "\\n";
            cerr << "Input is saved in input.txt\\n";
            return 0;
        }
    }
    cerr << "All tests passed\\n";
}
```


对拍生成器不要一开始就追求大数据。先覆盖： $n = 1$ 、全相同、完全逆序、只有一条边、重复边、无解、答案在端点、随机小数据。暴力程序必须比标准程序更简单，即使慢到 $O(n^3)$ 也可以；对拍的目标是正确，不是速度。

4.7 错题卡：以后每道题只记录一页

字段	记录内容
题目与提交时间	题号、语言、提交结果、使用的模板
最小错误输入	删除哪些元素后仍然能复现
错误类别	题意、边界、下标、溢出、状态、清空、复杂度、输出
被破坏的不变量	例如“当前区间和应等于四个子区间之和”
错误代码	只贴最小的 3-10 行，不要整页复制
修复动作	改了哪一个条件、初始化或数据结构
回归样例	输入、正确输出、以后什么时候重跑

真正有价值的错题记录不是“这题用了线段树”，而是“我把整段懒标记下传前直接访问了子区间；最小反例是先改全区间再查中间一格”。后者可以迁移到下一道题，前者只能记住一道题。

4.8 提交前五分钟固定动作

1. 用题面约束重新确认 `int`、`long long`、`__int128` 和浮点类型；
2. 手算一组 $n = 1$ ，一组重复值/重边，一组无解或不可达；
3. 检查 `solve()` 是否完整读完当前测试，所有全局状态是否清空；
4. 检查所有区间端点是闭区间还是半开区间，下标是 0-based 还是 1-based；
5. 检查输出顺序、大小写、精度和无解格式；
6. 如果仍然不放心，立刻写暴力和对拍，不要继续凭感觉改条件。

5 通用 RE 错误

RE 不像 WA 有输出可以对比——程序直接崩了，没有任何线索。排查思路：先定位崩在哪一行，再判断哪类原因。

5.1 RE 排查流程

1. 本地复现：把样例贴进来跑，看能不能在本地炸。
2. 加输出定位：在关键位置加 `cerr << "checkpoint 1" << endl;`，逐步缩小崩溃区间。
3. 极端数据： $n = 0$ 、 $n = 1$ 、 $n = N_{\max}$ 、全相同、全极值，单独测。
4. 开编译选项：本地编译加 `-fsanitize=address,undefined -g`，能直接报出越界和 UB 的行号。
5. 对照常见原因清单：逐条过下面的列表。

5.2 数组越界

- 开小了： $N = 10^5 + 10$ 但输入 n 可以到 2×10^5 。仔细看数据范围再开数组。
- 下标为负：`a[i - k]` 当 $i < k$ 时变负数。用前先判 `if(i >= k)`。
- 下标为 0：1-based 存图但 `head[0]` 没初始化；或反过来 0-based 但循环从 1 开始漏了 `a[0]`。
- 邻接表边数：无向图每条边存两次，边数组至少开 $2M$ 。
- 线段树：节点数组至少 $4N$ （不是 $2N$ ）。
- 组合数/阶乘表：预处理到 N 但查询 $C(n, m)$ 时 n 可能到 $2N$ 。

5.3 栈溢出（递归过深）

- 链状图 DFS: $n = 10^5$ 的链递归深度 10^5 层，默认栈（1 MB / 8 MB）直接爆。
- 解决方案：
 - 改用 BFS / 手写栈模拟 DFS。
 - Linux 下加 `ulimit -s unlimited`（某些 OJ 支持）。
 - Windows 下编译加 `-Wl,--stack,268435456`（256 MB 栈）。
- 无限递归：忘记写 `return` / base case，或者 DFS 忘标 `vis[u] = true`，同一个节点反复进入。
- 树形 DP 忘跳父亲：`dfs(v, u)` 里没写 `if(v == fa) continue`，父子互相递归。

5.4 除以零 / 取模零

- a / b 或 $a \% b$ 当 $b = 0$ 时直接 RE。
- 常见场景：二分 $mid = (l + r) / 2$ 本身不会为零，但 n / mid 会在 $mid == 0$ 时炸。
- GCD 循环里 $a \% b$ ，如果 b 初始就是 0 则第一轮就崩。
- 解法：所有除法/取模前加 `if(b == 0)` 特判，或保证逻辑上 $b \neq 0$ 。

5.5 非法内存访问

- 空指针：链表/树结构访问 `nullptr->val`。
- 野指针：`delete` 后继续使用。竞赛中少用动态内存，优先用静态数组 + 内存池。
- VLA 过大：`int a[n]` 在栈上分配， $n = 10^6$ 就可能爆栈。改用全局数组或 `vector`。

5.6 STL 误用

- 空容器取元素：
 - `v.back()` / `v.front()` 在 `v.empty()` 时 UB/RE。
 - `s.top()` / `q.front()` 在空栈/空队列时崩。
 - 用前必须判 `!v.empty()`。
- 迭代器失效：

- vector 的 `push_back` 可能导致扩容，所有迭代器和指针失效。
- set/map 的 `erase` 后原迭代器失效，用 `it = s.erase(it)` 或先 `next(it)`。
- 越界访问：`v[i]` 不做边界检查（`v.at(i)` 会抛异常）。
- 比较器不满足严格弱序：`sort` 的 `cmp` 用 `<=` 会导致内部断言失败 → RE（不只是 WA!）。

5.7 整数未定义行为

- 有符号溢出：C++ 中有符号整数溢出是 UB，编译器可能生成崩溃代码。
- 移位越界：`1 << 31` 对 32 位 `int` 是 UB（符号位）；`1 << 64` 对 64 位也是 UB。
- 负数移位：`(-1) << k` 和 `x >> 负数` 都是 UB。
- 开 `-fsanitize=undefined` 可以在本地立刻暴露这些问题。

5.8 常见 RE 清单速查

提交 RE 后，按此表逐条排查：

1. 数组大小够吗？下标有没有负数或超上界？
2. 递归深度超过 10^4 了吗？有没有无限递归？
3. 有除法或取模吗？除数可能为 0 吗？
4. 有 `sort` 自定义比较器吗？是否满足严格弱序（`<` 而非 `<=`）？
5. 用了 `stack/queue/vector` 的 `top()/front()/back()` 吗？容器可能为空吗？
6. 用了 `1 << k` 吗？ $k \geq 31$ 时要写 `1LL << k`。
7. 有 VLA（`int a[n]`）吗？ n 大时改用全局数组。
8. 线段树开了 $4N$ 吗？并查集/倍增表第二维开够了吗？
9. 多测时上一组残留的大数组会不会越界访问？
10. 本地加 `-fsanitize=address,undefined -g` 编译，跑一遍样例。

RE 和 WA 的本质区别：WA 是逻辑错误，RE 是访问了不该访问的东西。绝大多数 RE 都是数组越界和栈溢出，优先查这两个。

6 图论与树

6.1 建图

- 无向图边数开 $2M$ （正反各一条）。
- 点属性开 N ，边属性开 M ，别搞混。
- 多测清空要按上一组的范围清，链式前向星 `cnt = 0`。
- 编号 0-based 还是 1-based，循环和起点要一致。

6.2 特殊图

- 不连通：没保证连通就外层 `for(i=1;i<=n;i++) if(!vis[i]) dfs(i)`。
- 重边：邻接矩阵 `g[u][v] = min(g[u][v], w)`。
- 自环：影响入度、度数、连通性。必要时 `if(u==v) continue`。
- 链：递归深搜会爆栈。菊花：暴力会退化成 $O(n^2)$ 。

6.3 DFS / BFS

- 无向树 DFS 传父亲：`dfs(u, fa)`，跳过 `if(v==fa) continue`。
- BFS 入队时立刻标 `vis[v]=true`，不要等出队。
- 找所有路径要回溯：离开前 `vis[u]=0`。

6.4 最短路

- Dijkstra 堆优化：出队判 `if(vis[u]) continue`。
- 有负权 $\Rightarrow$ Dijkstra 失效，用 Bellman-Ford。
- Floyd：初始化 `inf`，对角线 0，循环顺序 k, i, j 。
- 松弛防溢出：`if(dist[u]!=inf && dist[v]>dist[u]+w)`。

6.5 拓扑排序

- 跑完检查 `cnt==n`，不等说明有环。
- 要字典序最小 $\Rightarrow$ 用小根堆代替普通队列。

6.6 LCA / 树形 DP

- 倍增第二维开 $\lceil \log_2 N \rceil$ ，预处理外层枚举步长、内层枚举节点。
- 树形 DP：先 `dfs(v)` 再用 `dp[v]` 更新 `dp[u]`（后序）。
- 多测时在 `dfs(u)` 开头清当前点的 DP 状态。

图论一直 WA/RE 时：在测试数据里加孤立点、自环、重边、退化成链/菊花，很多隐藏 bug 会立刻暴露。

6.7 图论模型先判型

拿到图论题先不要急着敲模板，先把题面翻译成下面四个问题：

1. 边有没有方向？无向边必须加入两次；有向边不能反向乱加。
2. 权值范围是什么？只有 0/1 用 0-1 BFS，非负任意权值用 Dijkstra，有负权考虑 Bellman-Ford 或差分约束。
3. 题目问的是一条路径、所有点的可达性、连通块，还是删点删边之后的结构？对应的算法完全不同。
4. 图是否保证连通、无环、没有重边？题面没有保证的性质都不能默认。

最容易发生的误判是“看到图就写 DFS”。DFS 只能完成遍历，不能自动解决最短路、拓扑序、强连通分量或割点问题。先写出状态含义：例如 `dist[u]` 是源点到 u 的最短距离，`vis[u]` 只是“是否已经发现”，两者不是一回事。

6.8 连通块、0-1 BFS 与最小生成树

6.8.1 连通块：外层循环不能漏

只从 1 号点 DFS 只能得到 1 号点所在的连通块。要统计整个无向图的连通块，必须对所有点补一层外循环：

```

vector<vector<int>>> g;
vector<int> vis;

void dfs(int u) {
    vis[u] = 1;                // 先标记，避免无向边来回递归
    for (int v : g[u]) {
        if (!vis[v]) dfs(v);
    }
}

int count_components(int n) {
    int components = 0;
    for (int u = 1; u <= n; ++u) {
        if (!vis[u]) {
            ++components;
            dfs(u);            // 每次启动 DFS 就找到一个新连通块
        }
    }
    return components;
}

```

无向图的 DFS 如果只写 `if (v == fa) continue`，遇到重边时仍可能出错：两条平行边中只有一条是“父边”，另一条不能被跳过。需要区分边编号时传 `parent_edge`，不要只传父节点。

6.8.2 0-1 BFS：权值必须真的只有 0 和 1

边权为 0 或 1 时，双端队列可以代替优先队列。松弛后，走 0 边放到队首，走 1 边放到队尾。它的关键不在“用了 deque”，而在于队列顺序保证较小距离优先被处理：

```

struct Edge { int to, w; };      // w 必须是 0 或 1
const long long INF = (1LL << 60);

vector<long long> zero_one_bfs(int s, const vector<vector<Edge>>& g) {
    int n = (int)g.size() - 1;
    vector<long long> dist(n + 1, INF);
    deque<int> q;
    dist[s] = 0;
    q.push_front(s);

    while (!q.empty()) {
        int u = q.front();

```

```

    q.pop_front();
    for (auto e : g[u]) {
        if (dist[e.to] > dist[u] + e.w) {
            dist[e.to] = dist[u] + e.w;
            if (e.w == 0) q.push_front(e.to);
            else q.push_back(e.to);
        }
    }
}
return dist;
}

```

检查方式：把一条 0 边和一条 1 边交错放进图里，确认算法不会把“边数最少”误当成“权值和最小”；再故意放入权值 2，确认自己知道这时必须换 Dijkstra。

6.8.3 Kruskal: 最小生成树的三个返回值

Kruskal 不只要返回总权值，还经常要返回选了多少条边。若最后选边数不是 $n - 1$ ，图不连通，不能把当前总和当作一棵生成树：

```

struct DSU {
    vector<int> fa, sz;
    DSU(int n) : fa(n + 1), sz(n + 1, 1) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
    bool unite(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false; // 会形成环，不能选
        if (sz[x] < sz[y]) swap(x, y);
        fa[y] = x; sz[x] += sz[y];
        return true;
    }
};

long long kruskal(int n, vector<tuple<int, int, int>> edges) {
    sort(edges.begin(), edges.end(),
        [](auto a, auto b) { return get<2>(a) < get<2>(b); });
    DSU dsu(n);
    long long answer = 0;
    int used = 0;
    for (auto [u, v, w] : edges) {
        if (dsu.unite(u, v)) {
            answer += w;
        }
    }
}

```



```
        ++used;
    }
}
return used == n - 1 ? answer : -1; // -1 代表不存在生成树
}
```

6.8.4 强连通分量、割点和桥：最容易混淆的地方

- 强连通分量只用于有向图；割点和桥通常讨论无向图。看到 `low` 不要直接套同一份 Tarjan 代码。
- 无向图 Tarjan 处理重边时要跳过“父边编号”，不能写成只跳过 `v == fa`。
- 桥判断是 `low[v] > dfn[u]`；割点的根节点需要单独判断 DFS 子树数是否至少为 2。
- 有向图拓扑排序的结果数量小于 n ，说明有环；这不是“少访问了几个点”，而是题目约束与 DAG 假设冲突。

6.9 图论回归样例：四组就能抓住大多数板子错误

1. **单点图**： $n = 1, m = 0$ 。检查初始化、答案是否为 0、LCA 是否允许 `lca(1,1)`。
2. **不连通图**： $n = 5$ ，边为 $(1,2), (2,3), (4,5)$ 。连通块应为 2，点 1 到点 5 应不可达。
3. **重边和自环**：边为 $(1,2,7), (1,2,3), (2,2,0)$ 。最短路应选权值 3，拓扑排序不能把自环当成正常边。
4. **链和菊花**：一组是 $1 - 2 - 3 - \dots - n$ ，另一组是 1 连接所有其他点。分别检查递归深度和邻接表遍历是否退化。

图论调试时建议同时打印“点数、边数、可达点数、选中边数、拓扑结果长度”。这些量一旦和理论值不符，比只打印最终答案更容易定位是建图、遍历还是转移错了。

7 动态规划

7.1 初始化

- 求最大值 $\Rightarrow$ 初始化 $-\infty$ (不是 0, 收益可能为负)。
- 求最小值 $\Rightarrow$ 初始化 $+\infty$ 。
- 不可达状态 ($dp[j]==inf$) 不要参与转移, 否则 $inf+cost$ 溢出。
- ”前 i 个的最优” vs ”以第 i 个结尾的最优”: 定义不同, 答案位置不同。

7.2 遍历顺序

- 区间 DP: 外层枚举长度 len , 内层枚举左端点 L , 推出 R 。
- 0-1 背包: 容量倒序。完全背包: 容量正序。
- 树形 DP: 先递归儿子, 再转移父亲。

7.3 转移遗漏

- LIS 每个元素可以单独成为长度 1, 别忘初始化 $dp[i]=1$ 。
- 计数 DP 有多个否定条件 $\Rightarrow$ 容斥, 注意别重复减。
- 滚动数组: 每轮开头清空当前层, 注意 $+=$ 和 $=$ 的区别。

7.4 边界

- 转移含 $dp[i-2] \Rightarrow$ 循环从 $i=2$ 或 $i=3$ 开始, 手写 base case。
- 背包: $j - weight[i]$ 必须 ≥ 0 。
- 答案位置: 状态是”以 i 结尾” $\Rightarrow$ 答案是 $\max(dp[1..n])$ 。

7.5 先写状态三句话，再写循环

DP 最常见的错误不是代码语法，而是状态含义在代码中途偷偷变了。开写前强制写下三句话：

1. **状态：** `dp[i][j]` 精确表示什么，已经处理了哪些元素， i, j 是否包含当前位置。
2. **转移：** 最后一步是什么，从哪些更小的状态转移过来，是否会重复使用同一个物品或元素。
3. **答案：** 是 `dp[n][m]`、某一行的最大值，还是所有结尾状态的最大值。

例如“前 i 个数的最长上升子序列”和“以第 i 个数结尾的最长上升子序列”不是同一个状态。前者答案通常看 `dp[n]`，后者必须看 `max(dp[1..n])`；只要把这两个答案位置混用，就会在答案不以最后一个元素结尾的样例上出错。

写完转移后，用 $n = 0, 1, 2$ 手算一遍。若连空前缀、单元素和不可能状态都说不清楚，继续写大模板只会把错误藏得更深。

7.6 遍历顺序：是否会偷偷重复使用

背包的循环方向不是格式问题，而是在决定一个物品能否在同一轮被重复使用：

```
// 0-1 背包：每个物品最多使用一次，容量必须倒序
for (int i = 1; i <= n; ++i) {
    for (int j = capacity; j >= weight[i]; --j) {
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}

// 完全背包：每个物品可以使用多次，容量必须正序
for (int i = 1; i <= n; ++i) {
    for (int j = weight[i]; j <= capacity; ++j) {
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}
```

区间 DP 通常按区间长度从小到大；树形 DP 通常先算儿子再算父亲；依赖前一层的滚动数组必须先确认当前层是否还会读取旧值。一个非常有效的反例是：只有一个物品、重量刚好能放两次的容量。如果 0-1 背包输出了使用两次的价值，循环方向就错了。

7.7 不可达状态：不能把 `INF` 当成普通数字

最小值 DP 用正无穷初始化，最大值 DP 用负无穷初始化，但“无穷”只是标记，不是可以参与加法的真实数：

```
const long long INF = (1LL << 60);
vector<long long> dp(n + 1, INF);
dp[0] = 0;

for (int i = 1; i <= n; ++i) {
    if (dp[i - 1] == INF) continue; // 不可达状态不能继续转移
    dp[i] = min(dp[i], dp[i - 1] + cost[i]);
}
```

如果把不可达状态写成 0，求最小值时会错误地让它成为最优；如果直接计算 `INF + cost`，可能溢出成负数，之后的 `min` 会把这个负数当成答案。最大值 DP 同理，不能让 `-INF + value` 参与转移。

7.8 滚动数组：清空的是“当前层”

二维 DP 压成一维后，旧状态和新状态共用同一块内存。最常见的错误是上一轮的值残留，或者把本轮需要的 `+=` 写成覆盖：

```
vector<long long> old(m + 1), now(m + 1);
for (int layer = 1; layer <= n; ++layer) {
    fill(now.begin(), now.end(), NEG_INF); // 每轮先清当前层
    for (int j = 0; j <= m; ++j) {
        now[j] = old[j]; // 不选当前物品
        if (j >= w[layer] && old[j - w[layer]] != NEG_INF) {
            now[j] = max(now[j],
                          old[j - w[layer]] + value[layer]);
        }
    }
    swap(old, now);
}
```

计数 DP 的滚动数组还要注意加法顺序：每个来源状态可能贡献多次，不能因为看到一个来源就把目标状态清零。多测时，数组大小、组合数、前缀和和记忆化缓存都要在每组开始重新初始化，不能只改 `n`。

7.9 答案位置、路径恢复与计数取模

- 求“恰好装满”时，不能把所有容量都当成合法答案；只允许读取 `dp[capacity]`。

- 求“最多装多少”时，容量不必装满，答案通常是 $\max(dp[0..capacity])$ 。
- 需要输出方案时，必须在转移发生时记录前驱，或者在最终状态上反向判断；只输出最优值不等于完成题目。
- 计数 DP 每次加法和乘法都要及时取模；减法要先加模数，不能依赖 C++ 对负数取模的结果。
- 多个方案是否有顺序，要决定循环是“先物品后容量”还是“先容量后物品”；两种顺序可能分别统计组合和排列。

7.10 DP 最小回归样例

1. **全负收益**：数组 $[-5, -2, -7]$ ，最大值应为 -2 ，可以抓出把最大值 DP 错误初始化为 0 。
2. **答案不在最后**：数组 $[3, 1, 2]$ 的最长上升子序列长度为 2 ，若只输出最后一个状态，容易错误输出 1 。
3. **0-1 与完全背包**：一个物品重量 2 、价值 3 ，容量 4 ；0-1 背包答案为 3 ，完全背包答案为 6 。
4. **不可达容量**：重量只有 3 ，查询容量 1 和 2 ，必须输出题目规定的不可达标记，不能输出 0 或一个溢出的负数。
5. **多测残留**：第一组使用较大的容量并得到答案，第二组只有一个物品且容量为 0 ，检查答案数组和滚动数组是否真正清空。
6. **计数顺序**：硬币面值 $1, 2$ 、目标和 3 ，若统计组合答案为 2 ，若统计排列答案为 3 ；先确认题目问哪一种。

DP 怎么都调不对时：找最小样例，把 DP 表打印出来，手算一张表，对比第一处不同的格子——那里就是 bug。

8 数据结构

8.1 并查集

- 合并必须根对根: $fa[find(u)] = find(v)$ 。
- 初始化: `iota(fa+1, fa+n+1, 1)` 或 `for(i=1;i<=n;i++) fa[i]=i`。
- 统计连通块: 数 $find(i)$ 的种类, 或数 $fa[i]==i$ 的个数。

8.2 线段树

- Lazy tag: 区间加是累加 $tag[ls] += tag[u]$, 区间赋值才是覆盖。
- Pushdown: 传 tag $\rightarrow$ 更新子节点 tree 值 $\rightarrow$ 清父 tag。三步缺一不可。
- 区间长度: 左儿子用 $mid-l+1$ (节点范围), 不是查询边界 L。
- 越界返回: 最大值返回 $-\infty$, 最小值返回 $+\infty$, 求和返回 0。

8.3 二分

- 出现 $l = mid \Rightarrow mid$ 上取整 $mid = (l+r+1)/2$, 否则死循环。
- 防溢出: $mid = l + (r-l)/2$ 。
- 浮点二分: 固定循环 100 次, 别用 `while(r-l>eps)`。
- `check(mid)` 里的辅助数组每次都要清空。

8.4 双指针 / 滑窗

- 左指针不能越过右指针: `while(l<=r && ...)`。
- 最长区间: 修复合法后更新答案。最短区间: 即将破坏条件前更新。
- 分段统计漏最后一段 $\Rightarrow$ 加哨兵 `s += '#'` 或 `a[n+1] = inf`。

8.5 树状数组：下标、前缀和与差分

树状数组只维护“前缀可合并”的信息。最常见的单点加、区间和模板如下，所有下标都假定为 1-based:

```
struct Fenwick {
    int n;
    vector<long long> tree;
    Fenwick(int n) : n(n), tree(n + 1, 0) {}

    void add(int x, long long value) {
        for (; x <= n; x += x & -x) tree[x] += value;
    }

    long long prefix_sum(int x) const {
        long long result = 0;
        for (; x > 0; x -= x & -x) result += tree[x];
        return result;
    }

    long long range_sum(int l, int r) const {
        if (l > r) return 0;
        return prefix_sum(r) - prefix_sum(l - 1);
    }
};
```

add(0, value) 会在 $x += x \& -x$ 后永远停在 0，造成死循环；prefix_sum(n+1) 可能越界。离散化之后也必须把排名改成从 1 开始。

8.5.1 区间加、单点查：把差分藏进树状数组

对原数组做差分 $d_i = a_i - a_{i-1}$ 。给区间 $[l, r]$ 加 v ，只需 add(l, v)、add(r+1, -v)；查询位置 x 时求差分前缀和。测试时必须包含 $r = n$ ，否则 r+1 越界的错误永远不会出现。

8.6 单调栈与单调队列

8.6.1 下一个更大元素：栈内保存什么

维护一个从栈底到栈顶单调递减的下标栈。处理 a_i 时，所有小于当前值的元素都找到答案；相等元素是否弹出要由题目要求决定：求“严格更大”通常弹出 $\leq$ ，求“更大或相等”通常只弹出 $<$ 。

```
vector<int> next_greater(const vector<int>& a) {
    int n = (int)a.size();
    vector<int> answer(n, -1), st; // st 保存还没有答案的下标
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] < a[i]) {
            answer[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return answer;
}
```

8.6.2 滑动窗口最小值：队首才是答案

单调队列保存下标，队列中的值从小到大。每次先删掉窗口左侧的下标，再弹出队尾所有不可能成为最小值的元素，最后读队首：

```
vector<long long> window_min(const vector<long long>& a, int k) {
    deque<int> q;
    vector<long long> answer;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
        q.push_back(i);
        if (i >= k - 1) answer.push_back(a[q.front()]);
    }
    return answer;
}
```

窗口长度 $k = 1$ 、 $k = n$ 、数组全相等、数组单调递增和单调递减，是单调队列最有价值的五组测试。尤其要确认队列存的是下标而不是值，因为只有下标才能判断元素是否过期。

8.7 离散化与区间结构

当数值很大但只关心相对大小时，先复制、排序、去重，再用 `lower_bound` 找排名。离散化不是“把值除以一个数”，重复值必须映射到同一个编号；若题目有区间端点和坐标长度，还要判断是否需要加入相邻点之间的空隙。

```
vector<long long> values = all_values;
sort(values.begin(), values.end());
values.erase(unique(values.begin(), values.end()), values.end());
```



```
int rank_of(long long x) {  
    return int(lower_bound(values.begin(), values.end(), x) - values.begin()) + 1;  
}
```

只把出现过的端点压成连续编号，适合统计点；如果要计算覆盖长度， $x = 1$ 和 $x = 2$ 之间的长度不能凭空消失，常需要把端点、端点加 1 或相邻坐标都放入离散化数组。先写清楚“一个编号代表点还是代表小区间”。

8.8 数据结构回归样例

1. **树状数组**： $n = 1$ ，先加在 1，再查询 $[1, 1]$ ；随后查询空区间 $[2, 1]$ ，答案应为 0。
2. **单调结构**：数组 $[3, 3, 3]$ ，分别测试严格大于与大于等于；窗口 $k = 1$ 和 $k = n$ 都必须输出完整正确结果。
3. **线段树懒标记**：初始 $[1, 1, 1, 1]$ ，整段加 5 后查询中间区间，再单点修改并查询整段，专门逼出漏写 `push_down` 的问题。
4. **离散化**：输入值 $[10^9, 10^9, -10^9, 0]$ ，确认重复值排名相同，负数不会因为数组下标类型转换而错位。

9 博弈论

博弈题先确认三件事：双方是否轮流操作、是否完全信息、是否存在后手无法操作的终止状态。默认下面讨论“双方最优、不能操作者输”的正常玩法；题目若写了反常玩法、和棋或同时操作，不能直接套用。

9.1 必胜态与必败态

把一个局面记为状态 s 。若当前玩家能走到某个必败态，当前态就是必胜态；若所有后继状态都是必胜态，当前态就是必败态。石子游戏的记忆化模板如下：

```
vector<int> moves;           // 允许拿走的石子数
vector<int> memo;           // -1 未计算, 0 必胜, 1 必败

bool win(int stones) {
    if (stones == 0) return false; // 没有合法操作, 当前玩家输
    int& answer = memo[stones];
    if (answer != -1) return answer;
    answer = 0;                  // 先假设所有走法都救不了
    for (int take : moves) {
        if (take <= stones && !win(stones - take)) {
            answer = 1;          // 走到对手必败态
            break;
        }
    }
    return answer;
}
```

这里的返回值是“当前局面是否必胜”，不是“谁最终获胜”。最容易错的是把 `memo` 按“上一个人是否赢”来理解，或者忘记先给终止状态赋值。若状态不是石子数而是多个变量，先把所有变量编码成一个整数，或用哈希表记录状态。

9.2 Bash 游戏：先找模周期

每次最多拿 k 个石子时， n 个石子的状态满足：

- $n \bmod (k+1) = 0$ 是必败态;
- 其他状态是必胜态, 拿走 $n \bmod (k+1)$ 个即可把对手送回必败态。

边界一定要包含 $n = 0$ 、 $n = k$ 、 $n = k+1$ 、 $n = 2(k+1)$ 。当允许拿走的集合不是连续的 1 到 k 时, Bash 公式通常失效, 应先用必胜必败 DP 打表, 再观察是否出现周期。

9.3 普通 Nim: 异或和为零就是必败

有若干堆石子, 每次从一堆拿任意正数。令

$$X = a_1 \text{ xor } a_2 \text{ xor } \cdots \text{ xor } a_n.$$

$X = 0$ 时是必败态; $X \neq 0$ 时必胜。构造操作不能只输出“存在”, 而要找出一堆:

```
long long xo = 0;
for (long long x : a) xo ^= x;

for (int i = 0; i < n; ++i) {
    long long target = a[i] ^ xo;
    if (target < a[i]) { // 只能减少, 不能把这一堆变大
        cout << i << ' ' << a[i] - target << '\n';
        break;
    }
}
```

恢复答案的检查式是 $a[i] \oplus xo < a[i]$ 。如果异或之后更大, 就不是合法的“拿走石子”。测试 $[0, 0]$ 、 $[1]$ 、 $[1, 1]$ 、 $[1, 2, 3]$ 和含有大于 2^{31} 的数, 分别检查全零、单堆、必败、异或抵消和整数类型。

9.4 Sprague-Grundy 函数与 mex

对无环有向状态图, 定义 $SG(s)$ 为所有后继状态的 Grundy 数集合的 mex, 即“不在集合中的最小非负整数”。终止状态的后继集合为空, 所以 $SG = 0$:

```
vector<vector<int>>> graph;
vector<int> sg;

int get_sg(int u) {
    int& result = sg[u];
    if (result != -1) return result;
    vector<int> seen(graph[u].size() + 2, 0);
    for (int v : graph[u]) {
        int value = get_sg(v);
        seen[value] = 1;
    }
    for (int i = 0; i < seen.size(); ++i) {
        if (!seen[i]) return i;
    }
    return -1;
}
```

```

    if (value < (int)seen.size()) seen[value] = 1;
}
result = 0;
while (result < (int)seen.size() && seen[result]) ++result;
return result;
}

```

多个互不影响的游戏的总 Grundy 数是各自 SG 值的异或。判断总局面时只需检查异或和是否为 0；若要恢复操作，就枚举某一子游戏的合法后继，寻找使总异或变为 0 的那个后继。

SG 记忆化搜索要求状态图无环，或者至少要先处理环。对有环图直接递归会无限递归；对“走过的点不能再走”的游戏，状态通常必须包含访问集合，不能只用当前顶点作为记忆化键。

9.5 反常 Nim、阶梯 Nim 与常见变形

- 反常 Nim 的终止规则相反：拿走最后一颗的人输。若所有堆都为 1，奇数堆是必败；否则仍检查异或和是否为 0。
- 阶梯 Nim 中，石子只能向更高或更低的相邻阶移动时，先确认题目规则；常见版本的关键量是奇数层堆的异或，而不是所有堆直接异或。
- “每次只能拿固定几种数量”通常是 SG 或必胜必败 DP；“可以从任意一堆拿任意数量”才是普通 Nim。
- 看到“最后一步得分”“操作次数限制”“有资源消耗”，先检查是否已经不是纯组合游戏，可能要把轮次或资源加入状态。

9.6 Minimax 极大极小搜索与 Alpha-Beta 剪枝

棋盘状态较小、双方都能看到完整局面时，可以枚举博弈树。下面模板返回“当前玩家从该状态能得到的最大得分”，其中轮到最大化玩家时取最大值，轮到最小化玩家时取最小值：

```

int minimax(State& state, int depth, int alpha, int beta, bool maximizing) {
    if (depth == 0 || terminal(state)) return evaluate(state);

    if (maximizing) {
        int best = INT_MIN;
        for (Move move : legal_moves(state)) {
            apply(state, move);
            best = max(best, minimax(state, depth - 1,
                                    alpha, beta, false));
        }
    }
}

```

```

        undo(state, move);
        alpha = max(alpha, best);
        if (alpha >= beta) break; // beta 剪枝
    }
    return best;
} else {
    int best = INT_MAX;
    for (Move move : legal_moves(state)) {
        apply(state, move);
        best = min(best, minimax(state, depth - 1,
                                alpha, beta, true));

        undo(state, move);
        beta = min(beta, best);
        if (alpha >= beta) break; // alpha 剪枝
    }
    return best;
}
}

```

调用前必须保证 `apply` 与 `undo` 完全对称；否则深层搜索结束后，父层状态已经被污染。若同一局面会从不同路径到达，可以把局面编码成字符串或整数做哈希去重，但哈希键必须包含轮到谁、剩余深度和所有影响后续决策的信息。

9.7 博弈题的现场检查顺序

1. 先写出“不能行动时谁输”，确认是正常玩法还是反常玩法。
2. 画出 0, 1, 2, 3 个石子的状态，手算前四个必胜必败态。
3. 状态无环且子游戏独立，优先考虑 SG；棋盘小且需要选择具体走法，考虑 Minimax。
4. 需要输出操作时，不能只判断胜负，必须验证操作后状态确实满足目标不变量。
5. 多测时清空 SG、记忆化数组、棋盘哈希和搜索计数器；尤其不要把上一组的周期结论直接带到下一组。

9.8 博弈回归样例

1. **必胜必败 DP**：允许拿 1 或 3 个，测试 $n = 0, 1, 2, 3, 4$ ，答案依次为败、胜、胜、胜、败。
2. **Bash**： $k = 2$ 时测试 $n = 0, 1, 2, 3, 4, 6$ ，所有 $n \bmod 3 = 0$ 的状态应为败。
3. **普通 Nim**： $[0, 0]$ 、 $[1]$ 、 $[1, 1]$ 、 $[1, 2, 3]$ ，分别检查零堆、单堆、异或为零和多堆抵消。

4. **反常 Nim:** 测试全是 1 的偶数堆和奇数堆, 再测试含有大于 1 的堆, 确认没有把普通 Nim 公式无条件套用。
5. **搜索状态:** 棋盘只有一个格子、无合法操作、只有一个合法操作、同一状态两条路径到达, 检查终止判断、回滚和状态去重。

博弈题不要一上来背结论。先用小数据把胜负序列写出来: 如果序列出现周期, 再证明周期; 如果是多个独立部分, 再验证操作是否真的只改变其中一部分。结论和样例对不上时, 优先怀疑游戏规则理解错, 而不是先改代码。

10 数学与精度

10.1 取模（再强调）

- 减法： $(a - b \% \text{MOD} + \text{MOD}) \% \text{MOD}$ 。
- 除法：乘逆元，不能直接除。
- 连乘：步步取模。

10.2 组合计数

- ”至少包含 A 或 B” $\neq$ 包含 A 的 + 包含 B 的（重复算了交集）。用容斥或正难则反。
- 组合数函数开头：`if(m<0 | m>n) return 0|`，保证 `fact[0]=1`。
- 球相同/不同、盒相同/不同、允不允许空——四个条件决定公式。

10.3 数论

- LCM 防溢出： $a / \text{gcd}(a,b) * b$ （先除后乘）。
- 分解质因数：循环结束后 `if(n>1)` 说明剩下的是一个大质因数。
- 负数整除：C++ 向零取整，数学向下取整，负数时结果不同。

10.4 浮点

- 整数幂不用 `pow`，用快速幂。
- 判完全平方：`long long r = round(sqrt(n)); if(r*r==n)`。
- double 判等：`abs(a-b) < 1e-9`。
- 分式比大小：交叉相乘 $a \cdot d$ vs $b \cdot c$ （注意分母符号和溢出）。

11 计算几何

11.1 精度

- 浮点比较必须用 eps: $\text{abs}(a-b) < 1e-9$, 直接 `==` 几乎必错。
- eps 取多少? 一般 10^{-9} 够用, 但坐标范围大 (10^9) 时误差会放大, 考虑用 10^{-7} 或整数化。
- 能用整数就不用浮点: 叉积、点积、距离平方都可以保持整数运算。
- `atan2` 精度有限, 角度排序优先用叉积比较。

11.2 向量基础

- 叉积 $\vec{AB} \times \vec{AC} = (B - A) \times (C - A)$, 正值 C 在 AB 左侧, 负值右侧, 零共线。
- 点积 $\vec{AB} \cdot \vec{AC}$: 正为锐角, 负为钝角, 零为直角。
- 向量旋转 90° : $(x, y) \rightarrow (-y, x)$ (逆时针)。
- 单位化: 除以模长, 注意模长为 0 的退化情况。

11.3 线段与直线

- 线段相交判定: 跨立实验 (两次叉积符号相反) + 共线时的投影重叠检查。
- 共线特判容易漏: 两线段共线且有重叠部分也算相交。
- 点在线段上: 叉积为 0 且点积 ≤ 0 (即投影在线段范围内)。
- 直线交点: 用叉积公式 $t = \frac{(C-A) \times d_2}{d_1 \times d_2}$, 分母为 0 说明平行。

11.4 凸包

- Andrew 算法排序后分上下凸壳，注意最后一个点不要重复加入。
- 叉积判断用 ≤ 0 还是 < 0 ：允许共线点在凸包上用 $<$ ，不允许用 $\leq$ 。
- 所有点共线时凸包退化成线段，面积为 0，周长要特判。
- $n \leq 2$ 时直接特判，不要跑凸包算法。

11.5 多边形

- 面积（Shoelace 公式）： $S = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|$ ，下标取模。
- 点在多边形内：射线法（奇偶计数）或转角法。射线法注意射线恰好过顶点的边界情况。
- 多边形方向：叉积求面积为正 $\Rightarrow$ 逆时针，为负 $\Rightarrow$ 顺时针。题目要求特定方向时先判断再 reverse。
- Pick 定理（格点）： $S = I + \frac{B}{2} - 1$ ，边上格点数 $= \gcd(|\Delta x|, |\Delta y|)$ 。

11.6 圆

- 两圆关系：比较圆心距 d 与 $r_1 \pm r_2$ ，注意内含/外切/相交/内切/外离五种。
- 直线与圆交点：圆心到直线距离 vs 半径，距离用叉积除以方向向量模长。
- 三点确定圆（外接圆）：解方程组或用垂直平分线交点，注意三点共线无解。
- 最小覆盖圆：随机增量法，期望 $O(n)$ ，注意先 `random_shuffle`。

11.7 旋转卡壳

- 求凸包直径（最远点对）：对踵点单调移动，不要每条边都从头扫。
- 叉积比较面积来移动对踵点：`cross(hull[i+1]-hull[i], hull[j+1]-hull[j]) > 0` 时 j 前进。
- 凸包点数 ≤ 2 时直接算距离，不要进旋转卡壳。

11.8 半平面交

- 有向直线：左侧为合法区域，方向一致才能排序。

- 按极角排序，平行的保留最左边那条。
- 双端队列维护：队首队尾都可能弹出，最后还要用队首检查队尾。
- 结果为空 $\Rightarrow$ 无解；结果无界 $\Rightarrow$ 加一个大矩形边界。

11.9 常见坑总结

- 坐标范围 10^9 时叉积会到 10^{18} ，必须用 `long long`。
- 三点共线、线段端点重合、多边形退化——这些边界情况是计算几何 WA 的重灾区。
- 排序比较函数里用叉积时，共线情况要用距离做二级排序，否则凸包/极角排序会乱。
- 输出精度：题目要求保留几位小数就用 `fixed << setprecision(k)`，别用默认格式。

12 读题错误

12.1 数值范围

- 变量可以是负数或 0 吗？ $-10^9 \leq a_i \leq 10^9$ 时别默认正数。
- 负数会破坏单调性、双指针的前提、乘法的符号。

12.2 核心对象

- **Permutation vs Array**: 排列意味着互不相同、值域 $1 \sim N$ ，可以用位置映射和置换环。
- **简单图**: 没写”no self-loops and multiple edges”就默认有。
- **有向 vs 无向**: 看清 directed/undirected, ”关注”是单向, ”朋友”是双向。

12.3 操作规则

- **同时 vs 依次**: simultaneously $\Rightarrow$ 开新数组存下一状态, 统一修改。
- **代价**: 是每删一个花 X , 还是执行一次操作总共花 X ?
- **Any vs All**: all 要求全部满足, any 只要存在一个。

12.4 坐标与格式

- 输入是 (row, col) 还是 (x, y) ? C++ 数组是 `grid[row][col]`。
- 输入顺序: 不一定先 N 后 M , 看清楚。
- 无解输出: 回题面 Output 段确认, 别凭经验写 -1 。